!!!Work in progress. Do not distribute!!!

Spatial Linking with `sora::`CHEATSHEET

Basics

sora can run in a single function.

sora is designed for spatially linking a geocoded dataset (**dataset**) to another geospatial dataset (**link_to**). The goal is to request and extract information from the geospatial dataset and add it to the observations of the geocoded dataset. This job ist done using the SoRa Geolinking API.

```
id            geometry                 id         area  mean count
<chr>      <POINT [m]>                 <chr>     <dbl> <dbl> <int>
1    (4106850 3096813)                 1     78036129.  71.5    81
2    (4583803 3108944)                 2     78036129.  61.2    79
3    (4551001 3272856)                 3     78036129.  76.1    78
```

Users have to define these two data sources and a spatial method to link them.

```
linking_job <-
  sora_request(
    dataset,
    link_to,
    method,
    ...
  )
```

https://sora.gesis.org/public/sora-user-mod/users/request-api-key Get an API key here

sora_available() Checks if the SoRa Geolinking API can be reached (requires an API key)

sora_job_status(linking_job) Checks the status of the job defined in **sora_request()**

Data

Define both the geocoded dataset and the geospatial dataset. Note not all of them are available off-site.

PRE-DEFINED DATASET

```
sora_request(dataset = <dataset_id>, ...)
```

Pre-defined datasets stem from SoRa partners providing georeferenced survey datasets. The SoRa datapicker gives an overview to the currently available data and the IDs needed for the argument **dataset**:

```
sora_datapicker("provided")
```

Users have to request access at the corresponding data providers. Usually, they have to conduct the linkages based on the original, sensitive data on-site via the Private API. However, in cases where synthetic data are available (note the string **synth** in the IDs), linkages can be tested and prepared off-site via the Public API. For a more fine-grained control over **<dataset_id>**, e.g., when aiming to filter specific cases one can exploit:

```
sora_provided(
  c(<ids>, survey = <dataset_id>)
)
```

sora_job_results(linking_job) Returns the results of the job defined in **sora_request()**

sora_simulate() Test run of **sora_request()**

sora_job_done() TRUE/FALSE depending on if job has finished

CUSTOM DATASET

```
sora_request(
  dataset = sora_custom(coords),
  ...
)

sora_request(
  dataset = sora_admin(codes),
  ...
)
```

sora can also be used with users' own custom data. If **coords** is not an **sf** object, a coordinate reference system has to be specified by entering an EPSG code:

```
sora_custom(coords, crs = <epsg_code>)
```

If **codes** denotes administrative boundary codes (AGS or zip codes), they also have to be specified using the codes from **sora_datapicker("admin")**:

```
sora_admin(codes, schema = <schema_code>)
```

https://sora.gesis.org/public/datapicker/ SoRa Online Data Picker

DATASET TYPES

Users can either work with

Geocoded points data, i.e., coordinates

Administrative boundaries, i.e., polygons

Depending on the choice the linking methods and selection areas may vary.

GEOSPATIAL DATA

```
sora_request(..., link_to = <dataset_id>)
```

Geospatial data in **sora** are pre-defined (mostly raster data). The SoRa datapicker gives an overview to the currently available data and the IDs needed for the argument **link_to**:

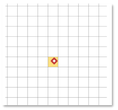
```
sora_datapicker("spatial")

sora_spatial(<dataset_id>)
```

Methods

Specify how the information of the geospatial data are processed.

LOOKUPS

```
sora_request(..., method = "lookup")
```



Extracts the information from the geospatial data at each point of the location of the geocoded

COUNTING OBJECTS

```
sora_request(..., method = "count_objects")
```

Counts the number of objects of the geospatial data within a specific selection area.

AGGREGATING ATTRIBUTES

```
sora_request(
  ...,
  method = "aggregate_attributes"
)
```

Aggregates the information from the geospatial data for a specific selection area by applying a descriptive statistic on the attributes within the area.

CALCULATING AREAS

```
sora_request(..., method = "area_polygons")
```

Calculates the size of a specific selection area of the geospatial data.

CALCULATING LENGTHS OF LINES

```
sora_request(..., method = "length_lines")
```

Calculates the length of lines in a specific selection area of the geospatial data.

CALCULATING DISTANCES

```
sora_request(
  ...,
  method = "nearest_poi",
  routing_type =
    <"fastest_route", "shortest_route">,
  transport_mode =
    <"foot-walking", "cycling-regular",
    "driving-car">
)
```

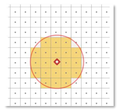
Calculates the fastest or shortest route to points of interest.

Selection Area

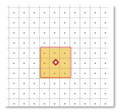
DIFFERENT AREA TYPES

Depending on the selection area the outcomes of the linking methods vary.

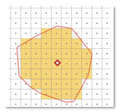
```
sora_request(
  ...,
  selection_area = <area_type>
)
```



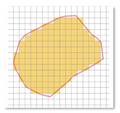
```
selection_area = "circle",
radius =  <1-20000 meters>
```



```
selection_area = "square",
length =  <1-20000 meters>
```



```
transport_mode = <"foot-
walking", "cycling-regular",
"driving-car">, routing_type
= <"time", "distance">,
interval = <1-45 minutes, 1-
10000 meters>
```



Specified within **sora_admin()**

Output

DESCRIPTIVE STATISTICS

Depending on the methods and the arguments here the output varies.

```
sora_request(
  ...,
  output = <output_specification>
)
```

min	Minimum value in selection area
max	Maximum value in selection area
mean	Mean value in selection area
median	Median value in selection area
sd	Standard deviation value in selection area
sum	Sum value in selection area
mode	Mode value in selection area

By default, **sora** outputs the size of the selection area as well as the number of features within the selection in the object returned from **sora_request()**.

Read the docs!

http://soraplus.git.gesis.org/sora-r/, e.g., for setting up your own API key and other vignettes!

GESIS Leibniz-Institut für Sozialwissenschaften

Leibniz-Institut für ökologische Raumentwicklung

SOEP Das Sozio-oekonomische Panel

https://sora-service.org/

BONUS TIP Wrap the Methods and Selection Area specification in a **sora_linking()** object for more clarity in coding. These objects include all arguments necessary for the linkage.